

Finansinis intelektas: 13 paskaita

Automatizuotos prekybos sistemos I: logika, architektūra ir vykdymas

dr. Mindaugas Juodis

2026 m. Gegužė

Vilniaus universitetas | Duomenų mokslas

Kur esame kurse?

- Jau perėjome tris svarbias grandis:
 - blockchain duomenų analizę,
 - DeFi biržų ir AMM matematiką,
 - skolinimo protokolus, likvidacijas ir Flash Loans.
- Iki šiol nagrinėjome mechanizmus atskirai.
- Dabar pereiname prie klausimo:
 - kaip visa tai sujungti į vieną veikiančią sistemą?
- Būtent tai ir yra automatizuota prekybos sistema.

Kodėl automatizuotos prekybos sistemos svarbios

- Iki šiol nagrinėjome atskiras temas:
 - duomenų analizę,
 - rizikos sąvokas,
 - DeFi mechanizmus.
- Automatizuota prekybos sistema leidžia šiuos elementus matyti vienoje struktūroje.
- Tokia sistema paprastai apima:
 - duomenų gavimą,
 - signalų formavimą,
 - rizikos kontrolę,
 - pavedimų vykdymą,
 - rezultatų stebėseną.
- Todėl ši tema natūraliai apibendrina kelias ankstesnes kurso dalis.

Ką realiai daro automatizuota prekybos sistema?

- Automatizuota prekybos sistema stebi rinkos duomenis ir pagal nustatytas taisykles priima veiksmą.
- Ji gali:
 - fiksuoti naujas kainas,
 - atpažinti signalą,
 - apskaičiuoti pozicijos dydį,
 - išsiųsti pavedimą,
 - stebėti, kas įvyko po jo.
- Pavyzdžiui, sistema gali pirkti, kai indikatorius rodo įėjimo signalą, ir uždaryti poziciją, kai pasiekiamas nuostolio limitas.
- Todėl automatizacija apima ne vien prognozę, bet ir visą vykdymo procesą.

Duomenys → Signalas → Rizikos kontrolė →
Vykdymas → Stebėsena

- Duomenys: kaina, apimtis, order book, on-chain rodikliai.
- Signalas: taisyklė, indikatorius arba modelis.
- Rizikos kontrolė: pozicijos dydis, stop-loss, ekspozicijos ribos.
- Vykdymas: pavedimo tipas ir ryšys su birža per API.
- Stebėsena: rezultatų tikrinimas ir sistemos būsenos sekimas.

Kokius duomenis gali naudoti automatizuota sistema?

- Kainos duomenys:
 - open, high, low, close, volume.
- Rinkos mikrostruktūros duomenys:
 - bids, asks, spread, order book imbalance.
- Blockchain ir DeFi duomenys:
 - active addresses,
 - exchange inflows ir outflows,
 - TVL,
 - liquidation events,
 - lending utilization.
- Papildomi duomenų šaltiniai:
 - social media sentiment,
 - naujienų tonas,
 - makroekonominiai rodikliai.

Ryšys su ankstesnėmis paskaitomis

- 11 paskaitoje aptarėme:
 - DEX veikimą,
 - AMM formulę,
 - slippage,
 - arbitražo logiką,
 - impermanent loss.
- 12 paskaitoje aptarėme:
 - lending pools,
 - utilization ratio,
 - health factor,
 - liquidation mechaniką,
 - flash loans.
- Automatizuotose sistemose šie reiškiniai tampa stebimais kintamaisiais, signalais arba rizikos indikatoriais.

Kaip sistema priima sprendimą?

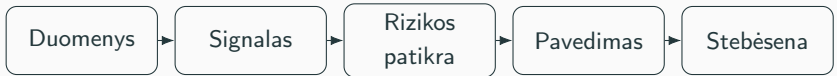
- Sistema veikia pagal iš anksto aprašytas taisykles.
- Paprasčiausiu atveju naudojama viena loginė sąlyga:
 - jei $RSI < 30$, generuojamas pirkimo signalas;
 - jei $RSI > 70$, generuojamas pardavimo signalas.
- Sudėtingesnėse sistemose jungiami keli informacijos šaltiniai:
 - techniniai indikatoriai,
 - on-chain metrikos,
 - sentimentas,
 - rizikos apribojimai.
- Todėl automatizavimas prasideda nuo aiškiai suformuluotų taisyklių, o ne nuo techninio ryšio su birža.

Kodėl vieno signalo dažnai nepakanka?

- Tas pats signalas skirtingomis rinkos fazėmis gali veikti nevienodai.
- Dėl to praktikoje dažnai naudojami:
 - keli signalai vienu metu,
 - papildomi filtrai,
 - platesnis rinkos kontekstas.
- Pavyzdžiui:
 - RSI gali rodyti perpardavimą,
 - tačiau tuo pačiu metu liquidation srautai gali dar didėti.
- Tokiu atveju sprendimas atidedamas arba mažinama pozicija.
- Vienas signalas paprastai rodo galimą kryptį, bet ne visada pakankamai apibūdina situaciją.

- Prekybos sistemoje svarbus ne tik signalas, bet ir tai, kaip valdoma rizika.
- Dažniausiai kontroliuojami šie elementai:
 - pozicijos dydis,
 - maksimalus leistinas nuostolis,
 - stop-loss taisyklės,
 - portfelio ekspozicija,
 - koreliuotų pozicijų kiekis.
- Tas pats signalas gali duoti skirtingą rezultatą priklausomai nuo pozicijos dydžio ar naudojamo sverto.
- Dėl to rizikos valdymas yra viena svarbiausių automatizuotos sistemos dalių.

Nuo signalo iki pavedimo



Pastaba

Problemos gali atsirasti bet kuriame etape: signalas gali būti klaidingas, rizikos filtras gali būti per silpnas, o pavedimas gali būti įvykdytas blogesne kaina nei buvo tikėtasi.

Pavedimų tipai

Market order

Vykdomas iš karto už geriausią tuo metu prieinamą kainą.

Tinka, kai svarbiausia:

- greitis,
- garantuotas įvykdymas,
- reakcija į staigų judėjimą.

Limit order

Vykdomas tik pasiekus nurodytą kainą arba geresnę.

Tinka, kai svarbiausia:

- kainos kontrolė,
- mažesni mokesčiai,
- darbas ramesnėje rinkoje.

	Market	Limit
Įvykdymo tikimybė	didelė	mažesnė
Kainos kontrolė	maža	didesnė
Slippage rizika	didesnė	mažesnė

Apibrėžimas

Automatizuota prekybos sistema (angl. *Automated Trading System*, arba *Trading Bot*) blockchain kontekste yra kompiuterinė programa, kuri tiesiogiai jungiasi prie kriptovaliutų biržų per API sąsają arba prie decentralizuotų protokolų (DEX) ir vykdo pirkimo bei pardavimo sandorius pagal iš anksto nustatytas taisykles.

- Sistema gali veikti centralizuotose biržose arba decentralizuotoje aplinkoje.
- Jos logika remiasi aiškiai aprašytais taisyklėmis, indikatoriais arba modeliais.
- Praktikoje ji naudojama prekybai, arbitražui, likvidumo valdymui ar kitoms automatizuotoms strategijoms.

API kaip ryšys tarp sistemos ir rinkos

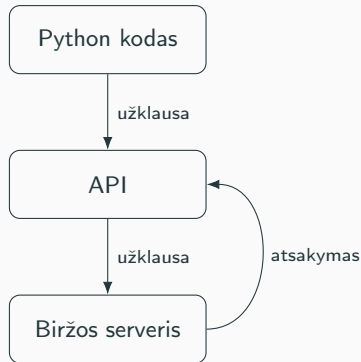
- Kad automatizuota sistema galėtų veikti, jai reikia būdo susisiekti su birža ar kitu duomenų šaltiniu.
- Šį ryšį užtikrina API – programinė sąsaja, per kurią galima:
 - gauti rinkos duomenis,
 - stebėti kainų pokyčius,
 - tikrinti balansą,
 - siųsti pavedimus.
- Praktikoje dažniausiai naudojami du pagrindiniai būdai:
 - REST API,
 - WebSocket.

Kas tiksliai yra API?

Bendra idėja

API (Application Programming Interface) yra taisyklių rinkinys, leidžiantis vienai programai keistis duomenimis su kita programa.

- Prekybos sistemos atveju API jungia jūsų kodą su birža ar duomenų tiekėju.
- Per API galima:
 - gauti kainas,
 - matyti order book,
 - tikrinti balansą,
 - siųsti pavedimus,
 - gauti pavedimų būseną.
- Be API automatizuota sistema negali veikti realioje rinkoje.



REST API: užklausa ir atsakymas

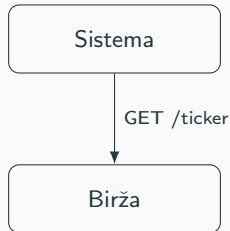
Kaip veikia

REST API dirba pagal principą:

request → response

Programa išsiunčia vieną užklausą ir gauna vieną atsakymą.

- Tinka tada, kai reikia konkretaus veiksmo:
 - gauti istorinę kainų seriją,
 - patikrinti balansą,
 - pateikti pavedimą,
 - atšaukti pavedimą.
- Paprasta įgyvendinti ir testuoti.
- Tačiau dažnas kartojimas gali didinti apkrovą ir delsą.



Pavyzdys

Sistema klausia:

/ticker/BTCUSDT

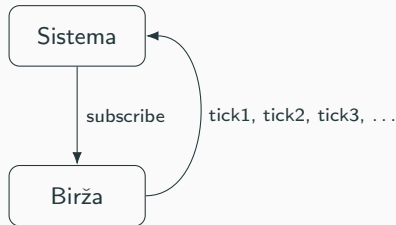
Birža grąžina: paskutinę kainą, bid, ask, apimtį.

WebSocket: nuolatinis duomenų srautas

Kaip veikia

WebSocket sukuria nuolatinį ryšį tarp sistemos ir serverio. Serveris gali siųsti naujus duomenis iš karto, kai tik jie atsiranda.

- Tai naudinga, kai reikia stebėti:
 - realaus laiko kainas,
 - sandorius,
 - order book pokyčius,
 - likvidacijų srautus.
- WebSocket mažina bereikalingų užklausų kiekį.
- Dėl to jis ypač tinka stebėsenai, o ne vienkartiniams veiksams.



Tai primena ne pavienius klausimus, o nuolatinį „kanalą“, kuriuo serveris transliuoja atnaujinimus.

REST API ir WebSocket: kada kuris naudojamas?

Užduotis	REST API	WebSocket
Istoriniai duomenys	tinka	dažniausiai nenaudojama
Balanso tikrinimas	tinka	paprastai nenaudojama
Pavedimo pateikimas	tinka	kartais, priklauso nuo biržos
Kainos stebėjimas realiu laiku	galima, bet neefektyvu	tinka geriau
Order book srautas	ribotai	tinka geriau

Praktinis principas

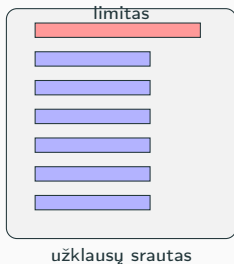
Daugelyje sistemų šie du būdai nėra konkuruojantys. Paprastai WebSocket naudojamas stebėti rinką, o REST API – konkreitiems veiksmams ir užklausoms.

Problema

Biržos riboja užklausų skaičių per minutę. Jei sistema siunčia per daug užklausų, ryšys gali būti sulėtintas arba laikinai užblokuotas.

Dažnas sprendimas

- mažinti nereikalingų REST užklausų kiekį,
- kainoms naudoti WebSocket,
- užklausas grupuoti ir planuoti.



API raktai ir prieigos kontrolė

Kas paprastai naudojama

- API key
- secret key
- leidimų apribojimai
- IP whitelisting

Ko paprastai vengiama

- raktų rašymo tiesiai kode
- withdrawal leidimų
- darbo be serverio apribojimų
- vieno rakto kelioms sistemoms

Technškai tai atrodo kaip smulkmena, bet praktikoje prastos prieigos teisės yra viena dažniausių sisteminių klaidų.

Kas yra sistemos būseną?

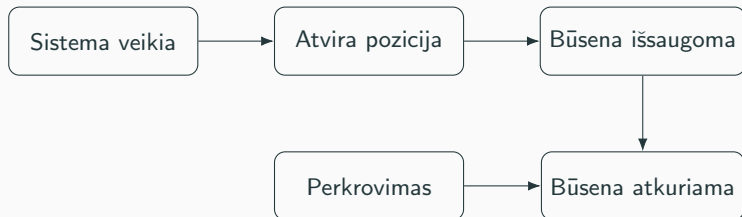
Apibrėžimas

Sistemos būseną yra informacija apie tai, ką sistema jau padarė ir kokioje padėtyje ji šiuo metu yra.

Būsenos elementas	Pavyzdys
Atvira pozicija	long 1 BTC
Aktyvus pavedimas	limit sell order
Rizikos taisyklė	stop-loss ties 58 000
Paskutinis signalas	pirkimo signalas 10:05

Kodėl tai svarbu

Be šios informacijos sistema po perkrovimo ar klaidos gali priimti sprendimus taip, tarsi pradėtų darbą nuo nulio.



Kodėl to reikia

Jei sistema nutrūksta ir neprisimena atviros pozicijos, ji gali neteisingai interpretuoti rinkos situaciją ir siųsti perteklinius pavedimus.

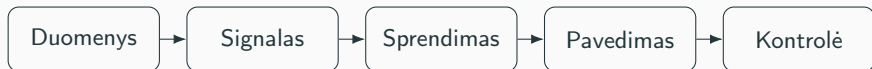
Kodėl sistemai reikia būsenos?

Pagrindinė idėja

Automatizuota sistema turi „atsiminti“, ką ji jau yra padariusi: ar turi atvirą poziciją, kokie aktyvūs pavedimai, kokie stop-loss lygiai.

- Jei po perkrovimo sistema neprisimena savo būsenos, ji gali:
 - pakartoti tą patį pavedimą,
 - neteisingai įvertinti riziką,
 - „manyti“, kad pozicijos nėra, nors ji iš tiesų atvira.
- Todėl būseną dažnai saugoma faile, duomenų bazėje arba atminties sistemoje.

Ką sistema daro po to, kai gauna duomenis?



- Vien gauti duomenis nepakanka.
- Sistema turi nuspręsti, ar signalas pakankamas veiksmui.
- Po pavedimo pateikimo dar reikia tikrinti, kas realiai įvyko.

Signalas ir vykdymas nėra tas pats

Signalas

Signalas nurodo, kad pagal taisyklės susidarė sąlygos pirkti, parduoti arba nieko nedaryti.

- Jis priklauso nuo duomenų ir modelio logikos.
- Tai dar nėra įvykęs sandoris.

Vykdymas

Vykdymas yra techninis etapas, kai sprendimas paverčiamas pavedimu rinkoje.

- Čia svarbūs API, delsos, pavedimo tipas, kaštai.
- Todėl teisingas signalas dar negarantuoja gero rezultato.

Du pagrindiniai nesėkmės šaltiniai

Modelio pusė

- silpnas signalas,
- netinkamos taisyklės,
- prastas pritaikymas rinkos fazei,
- per didelis pasitikėjimas istorija.

Vykdymo pusė

- per lėtas reagavimas,
- API klaidos,
- neįvertinti kaštai,
- bloga sistemos būseną po sutrikimo.

Svarbu mokėti atskirti, ar problema kilo sprendimo logikoje, ar techniniame realizavime.

Kur veikia tokia sistema?

Aplinka	Stiprybė	Ribojimas
Namų kompiuteris	lengva pradėti	nestabilus veikimas
VPS	nuolatinis darbas	reikia priežiūros
Docker	atkartojama aplinka	papildomas techninis sluoksnis

Praktinis principas

Akademiniu požiūriu svarbu suprasti, kad automatizuota sistema yra ne tik strategija, bet ir infrastruktūra, kurioje ta strategija turi veikti stabiliai.

Prieš realų paleidimą: paper trading

Kas tai yra

Sistema veikia realiu laiku, tačiau vietoje realių pinigų naudojama testinė arba simuliuojama aplinka.

- Tai tarpinis etapas tarp backtesting ir live trading.
- Čia tikrinama ne tiek pelno idėja, kiek sistemos nuoseklumas:
 - ar signalai atsiranda laiku,
 - ar pavedimai siunčiami teisingai,
 - ar veikia rizikos taisyklės,
 - ar sistema tinkamai reaguoja į klaidas.



Taigi dar karta kas yra backtesting?

Pagrindinė idėja

Backtesting yra strategijos tikrinimas istoriniuose duomenyse, siekiant įvertinti, kaip jos taisyklės būtų veikusios praeityje.

- Jis padeda suprasti strategijos logiką.
- Leidžia pamatyti signalų dažnį, grąžos dinamiką ir jautrumą rinkos fazėms.
- Tačiau geras istorinis testas dar negarantuoja gero veikimo ateityje.

Kodėl backtesting gali klaidinti?

Problema	Ką ji reiškia
Look-ahead bias	naudojama informacija, kurios tuo metu dar nebuvo
Survivorship bias	testuojama tik su „išgyvenusiais“ aktyvais
Overfitting Ignoruoti kaštai	taisyklės per daug pritaikytos istorijai nejtraukiami mokesčiai, delsa, vykdymo trintys

Pastaba

Istorinis testas gali atrodyti įspūdingai, bet būti menkai vertingas, jei jo prielaidos nėra realistiškos.

Look-ahead bias ir overfitting

Look-ahead bias

Strategija netyčia „pamato“ ateities informaciją ir todėl testas tampa nesąžiningai geras.

Pavyzdys

Jei sprendimas tariamai priimamas dienos viduryje, bet signalui naudojama dienos uždarymo kaina, gaunamas klaidingas pranašumas.

Overfitting

Sistema per daug tiksliai prisitaiko prie konkrečios istorijos, bet prastai veikia naujuose duomenyse.

Pavyzdys

Per daug parametų suderinta vienam periodui gali sukurti „gražų“ testą, kuris vėliau nepasikartoja.

Walk-forward principas



- Strategija kalibruojama viename lange ir tikrinama kitame.
- Tada langas pastumiamas į priekį.
- Taip geriau matyti, ar sistema išlieka stabili keičiantis rinkos sąlygoms.

Pagrindinė mintis

Automatizuota prekybos sistema gali turėti logišką strategiją, bet vis tiek neveikti taip, kaip tikėtasi realioje rinkoje.

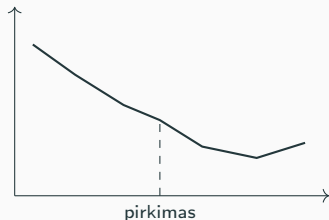
- Problemos dažnai kyla ne iš pačios idėjos, o iš jos įgyvendinimo.
- Tipiniai šaltiniai:
 - per lėtas vykdymas,
 - neįvertinti kaštai,
 - klaidingi duomenys,
 - netinkamai valdoma būseną,
 - strategijos pritaikymas ne tai rinkos fazėi.
- Toliau pažiūrėsime į kelis konkretesnius pavyzdžius.

Pavyzdys: RSI signalas ir per ankstyvas jėjimas

Situacija

Sistema turi taisyklę: jei $RSI < 30$, generuojamas pirkimo signalas.

- 10:00 val. RSI nukrenta iki 28.
- Sistema perka BTC.
- Tačiau po 20 minučių rinka toliau krenta dar 4%.



Interpretacija

Vien perpardavimo signalas dar nereiškia, kad kritimas baigėsi. Tokiose situacijose dažnai reikia papildomo filtro arba mažesnės pozicijos.

Pavyzdys: liquidation signalas dar nėra dugnas

Situacija

Sistema stebi liquidation srautus ir mato staigų jų šuolį.

- Idėja gali būti tokia: masinės likvidacijos reiškia paniką, todėl rinkai arti dugno.
- Tačiau praktikoje po pirmojo likvidacijų šuolio gali sekti antras ir trečias etapas.
- Todėl vien liquidation spike ne visada yra pirkimo signalas.

Laikas	Liquidations
09:00	12 mln.
10:00	54 mln.
11:00	88 mln.
12:00	41 mln.

Išvada

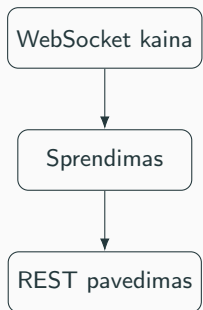
Likvidacijos dažniau veikia kaip konteksto indikatorius, o ne kaip savarankiškas sprendimo pagrindas.

Pavyzdys: API veikia, bet sistema vis tiek klusta

Situacija

Sistema gauna kainas per WebSocket ir siunčia pavedimus per REST API.

- WebSocket rodo naują kainą akimirksniu.
- Tuo metu pavedimo siuntimas per REST užtrunka ilgiau.
- Jei rinka juda greitai, sistema priima sprendimą jau pagal pasenusią kainą.



Išvada

Net kai visi komponentai „veikia“, jų tarpusavio delsa gali pakeisti rezultatą.

Pavyzdys: sistema „pamiršo“, kad jau turi poziciją

Situacija

Sistema nusipirko aktyvą, bet po perkrovimo neatsikūrė būsenos duomenys.

- Po restarto sistema „mano“, kad pozicijos nėra.
- Ji gauna naują signalą ir perka dar kartą.
- Rizika portfelyje padvigubėja, nors modelio logika to neplanavo.

Etapas	Pozicija
Iki restarto	1 BTC
Po restarto	„0 BTC“
Po naujo pirkimo	2 BTC

Išvada

Būsena nėra techninė smulkmena. Ji tiesiogiai susijusi su rizikos kontrole.

Pavyzdys: du signalai rodo skirtingas kryptis

Situacija

RSI rodo perpardavimą, todėl techninis signalas siūlo pirkti. Tuo pačiu metu exchange inflow rodiklis smarkiai auga, o tai gali reikšti didėjantį pardavimo spaudimą.

Klausimas

Ar sistema turi pirkti iš karto, ar laukti papildomo patvirtinimo?

Pamoka

Automatizuotoje sistemoje dažnai svarbu ne vienas signalas, o jų derinys ir prioritetai.

Pavyzdys: signalas atsirado, bet API atsakė per vėlai

Situacija

Sistema gauna pirkimo signalą 10:00:00.
Pavedimas išsiunčiamas 10:00:01, bet
patvirtinimas iš biržos grįžta tik 10:00:03.

- Per tas 2 sekundes rinka jau pajuda aukštyn.
- Sprendimas buvo teisingas, bet įvykdymas pablogino rezultatą.

Signalas $\neq$ rezultatas

Šis skirtumas ypač svarbus greitesnėse strategijose.

Situacija

Strategija buvo testuota laikotarpiu, kai rinka daugiausia judėjo šoniniu režimu. Sistema veikė stabiliai ir generavo nedidelį, bet nuoseklų pelną.

- Staiga įvyksta stiprus kritimas, ir volatilumas kelis kartus padidėja.
- Signalai pradeda dažnėti, pavedimai siunčiami dažniau, kaštai išauga.
- Strategija, kuri atrodė stabili, pradeda generuoti nuostolius.

Pamoka

Sistema turi būti vertinama ne tik pagal vieną istorijos režimą.

Pavyzdys: on-chain signalas yra naudingas, bet ne momentinis

Situacija

Sistema naudoja exchange outflow rodiklį kaip akumuliacijos požymį.

- Rodiklis iš tiesų gali būti informatyvus.
- Tačiau jis ne visada atnaujinamas taip greitai kaip kainos duomenys.
- Todėl trumpalaikėje prekyboje toks signalas gali vėluoti.

Pamoka

Ne visi geri signalai tinka visiems laiko horizontams.

Pavyzdys: sentimentas ne visada interpretuojamas teisingai

Situacija

Sistema analizuoja naujienų srautą ir aptinka žodžius „extreme“, „shock“, „collapse“.

- Vienu atveju tai gali reikšti neigiamą naujieną.
- Kitu atveju tai gali būti antraštė apie „shockingly good earnings“ arba ironija socialiniuose tinkluose.

Pamoka

NLP signalas dažnai yra naudingas, bet jautrus kontekstui ir teksto tonui.

Pavyzdys: stop-loss egzistuoja, bet rinka jį „peršoka“

Situacija

Sistema nustato stop-loss ties 58 000, tačiau rinkoje įvyksta staigus šuolis žemyn ir pirmas realus įvykdymas gaunamas ties 57 600.

- Taisyklė egzistavo.
- Tačiau rinka ne visada juda tolygiai.
- Todėl realus nuostolis gali būti didesnis nei planuotas.

Pamoka

Rizikos taisyklė yra būtina, bet ji nereiškia idealaus vykdymo.

Pavyzdys: dvi pozicijos atrodo skirtingos, bet rizika ta pati

Situacija

Sistema vienu metu perka BTC ir ETH, manydama, kad tai dvi atskiros pozicijos.

- Tačiau stipraus rinkos judėjimo metu šie aktyvai dažnai krenta kartu.
- Formali pozicijų įvairovė dar nereiškia tikros diversifikacijos.

Pamoka

Automatizuota sistema turi vertinti ne tik pozicijų skaičių, bet ir jų tarpusavio priklausomybę.

Pavyzdys: pelnas prieš kaštus ir pelnas po kaštų

Rodiklis	Prieš kaštus	Po kaštų
Vidutinė grąža per sandorį	0.35%	0.08%
Sandorių skaičius	220	220
Bendra išvada	atrodo gera	beveik nepelninga

Pamoka

Strategija gali atrodyti gera „ant popieriaus“, bet tapti silpna įtraukus mokesčius, slippage ir kitus vykdymo kaštus.

Pavyzdys: sistema gera akademiškai, bet neparuošta realiam naudojimui

Situacija

Modelis gerai prognozuoja kryptį istoriniuose duomenyse, bet sistema neturi:

- API klaidų apdorojimo,
- būsenos išsaugojimo,
- rizikos kontrolės po perkrovimo,
- testavimo realiu laiku.

Pamoka

Automatizuota prekybos sistema yra ne tik modelis, bet visa grandinė: duomenys, signalai, vykdymas, kontrolė ir testavimas.

Ką reiškia rizikos kontrolė praktikoje?

Elementas	Paskirtis
Pozicijos dydis	Riboti vieno sandorio poveikį portfeliui
Stop-loss	Apriboti nuostolį nepalankiu atveju
Take-profit	Fiksuoti rezultatą pagal taisyklę
Ekspozicijos ribos	Nekaupti per didelės pozicijos vienoje rinkoje
Koreliacijos kontrolė	Vengti kelių beveik vienodų statymų vienu metu

Pastaba

Rizikos kontrolė nėra atskiras priedas prie strategijos; ji yra sistemos dalis nuo pat pradžių.

Pavyzdžiai

- Kuo skiriasi REST API ir WebSocket?
- Kodėl backtesting rezultatai gali būti klaidinantys?
- Kuo skiriasi signalas nuo vykdymo?
- Kokią vietą automatizuotoje sistemoje užima rizikos kontrolė?
- Kodėl paper trading yra svarbus etapas?

Tokie klausimai tikrina ne programavimo detalę, o supratimą apie sistemos veikimo logiką.



- Automatizuota sistema jungia kelis sluoksnius, o ne vien techninį indikatorių.
- Geras istorinis testas dar nėra pakankama sąlyga pereiti į realų vykdymą.
- Praktikoje daug problemų atsiranda ne modelio, o vykdymo lygmenyje.

Kas bus toliau

Kitoje paskaitoje dėmesys pereis nuo sistemos infrastruktūros ir vykdymo prie pačių strategijų klasifikavimo ir jų kokybės vertinimo.

Šiandien	Kita paskaita
API, duomenų srautai, vykdymas	strategijų tipai
latency, backtesting ribos	performance metrics
paper trading, rizikos kontrolė	strategijų palyginimas

13 paskaita: automatizuotos prekybos sistemos I

Ačiū už dėmesį

Toliau – strategijų tipai, jų palyginimas ir pasiruošimas egzaminui.